

Azure AD Automated User Sync Tool

User Manual

System Prerequisites

- **Operating System:** Windows 10/11 or Windows Server
- **Python Environment:** Python 3.6 or higher installed
- **Dependencies:** requests and python-dotenv libraries
- **Permissions:** The Azure AD Application must have Group.ReadWrite.All and User.Read.All permissions with Admin Consent

Installation and Configuration

Step 1: Environment Setup

Create a dedicated directory for the automation tool. Inside this folder, you will place the Python script and the configuration file.

1. Open a terminal or command prompt
2. Install the required Python libraries by running the following command:

```
pip install requests python-dotenv
```

Step 2: Configuration File Setup

Create a file named `.env` in the same directory as the script. This file stores your credentials securely outside of the code. Open the file with a text editor and add the following lines, replacing the placeholders with your actual Azure AD application details:

```
CLIENT_ID=your_application_client_id
TENANT_ID=your_tenant_id
CLIENT_SECRET=your_application_client_secret
LOG_LEVEL=INFO
```

Step 3: Script Configuration

Open the `auto_sync_users.py` file in a text editor. Locate the `COUNTRY_GROUP_MAP` dictionary near the top of the script. Update this section to match your organization's country codes and the corresponding Security Group display names.

```
COUNTRY_GROUP_MAP = {
    "SG": "SG-Users-Security",
    "MY": "MY-Users-Security",
    "TH": "TH-Users-Security",
    "VN": "VN-Users-Security",
}
```

Automation Setup

To run the script automatically every 24 hours, use the Windows Task Scheduler.

Step 1: Create a Batch File

Create a file named `run_sync.bat` in the same directory. This file ensures the script runs from the correct folder path. Add the following content:

```
@echo off
cd /d "%~dp0"
python auto_sync_users.py
```

Step 2: Configure Task Scheduler

1. Press Win + R, type `taskschd.msc`, and press Enter
2. In the right-hand Actions pane, click **Create Basic Task**
3. **Name:** Enter "Azure AD User Sync"
4. **Trigger:** Select **Daily** and set the time (e.g., 02:00 AM)
5. **Action:** Select **Start a program**
6. **Program/script:** Browse and select the `run_sync.bat` file you created
7. **Start in: Crucial Step.** Enter the full path to the folder containing your script (e.g., `E:\ITProjects\AutoAddUsers`). This ensures the script can find the `.env` file and write logs.

Maintenance and Secret Rotation

Over time, you may need to update your Client Secret for security compliance. Follow these steps to rotate the key without breaking the automation.

Step 1: Generate New Secret in Azure Portal

1. Log in to the Azure Portal and navigate to **Microsoft Entra ID > App registrations**
2. Select your application
3. Go to **Certificates & secrets > Client secrets**
4. Click + **New client secret**. Add a description and expiration date
5. **Copy the Value** of the new secret immediately. Do not close the window yet

Step 2: Update the Environment File

1. Navigate to your script folder on the server
2. Open the `.env` file
3. Replace the old value next to `CLIENT_SECRET=` with the new secret value you just copied
4. Save the file

Step 3: Cleanup

1. Return to the Azure Portal
2. Delete the old, expired secret to ensure only the new key is valid
3. Manually run the `run_sync.bat` file once to verify the new credentials work correctly

Troubleshooting

If the automation fails, check the `logs` folder created in the script directory. The logs are named by month (e.g., `sync_log_202603.log`).

- **Authentication Failed:** Check if the `CLIENT_SECRET` in the `.env` file matches the active secret in Azure Portal
- **403 Forbidden:** Ensure the Azure AD Application has the correct API permissions and that Admin Consent has been granted
- **Group Not Found:** Verify that the group names in `COUNTRY_GROUP_MAP` match the "Display Name" in Azure AD exactly

Complete Script Content

```
# -*- coding: utf-8 -*-

"""
Azure AD 用户同步脚本 (Windows 计划任务专用版)
功能: 根据 License 状态自动添加/移除用户组成员
"""

import os
import sys
import json
import time
import logging
import requests
from datetime import datetime
from typing import Dict, List, Optional
from dotenv import load_dotenv

# ===== 日志配置 =====
# 确保 logs 目录存在
if not os.path.exists('logs'):
    os.makedirs('logs')

log_level_str = os.getenv("LOG_LEVEL", "INFO").upper()
log_level = getattr(logging, log_level_str, logging.INFO)

# 日志文件名格式: logs/sync_log_202603.log (按月归档)
log_filename = f"logs/sync_log_{datetime.now().strftime('%Y%m')}.log"

logging.basicConfig(
    level=log_level,
    format='%(asctime)s | %(levelname)-8s | %(message)s',
    handlers=[
        logging.FileHandler(log_filename, encoding='utf-8'),
        logging.StreamHandler(sys.stdout) # 同时也输出到控制台, 方便调试
    ]
)
logger = logging.getLogger(__name__)

# ===== 加载环境变量 =====
load_dotenv(dotenv_path='.env', verbose=True)

CLIENT_ID = os.getenv("CLIENT_ID")
TENANT_ID = os.getenv("TENANT_ID")
CLIENT_SECRET = os.getenv("CLIENT_SECRET")
SCOPE = "https://graph.microsoft.com/.default"

# 🇨🇳 国家代码到【纯安全组】的映射表
COUNTRY_GROUP_MAP = {
    "SG": "SG-Users-Security",
    "MY": "MY-Users-Security",
    "TH": "TH-Users-Security",
    "VN": "VN-Users-Security",
}

# 默认组 (可选)
```

```

DEFAULT_GROUP_NAME = None

# 全局配置
MAX_RETRIES = 3
RETRY_DELAY = 2

# 检查必要配置
def check_config():
    missing = []
    if not CLIENT_ID: missing.append("CLIENT_ID")
    if not TENANT_ID: missing.append("TENANT_ID")
    if not CLIENT_SECRET: missing.append("CLIENT_SECRET")

    if missing:
        logger.critical(f"❌ 环境变量缺失: {'', '.join(missing)}。请检查 .env 文件。")
        sys.exit(1)

check_config()

class AzureADSync:
    def __init__(self):
        self.token = None
        self.session = requests.Session()
        adapter = requests.adapters.HTTPAdapter(max_retries=MAX_RETRIES)
        self.session.mount('http://', adapter)
        self.session.mount('https://', adapter)

    def get_app_token(self) -> Optional[str]:
        url = f"https://login.microsoftonline.com/{TENANT_ID}/oauth2/v2.0/token"
        data = {
            "client_id": CLIENT_ID,
            "client_secret": CLIENT_SECRET,
            "scope": SCOPE,
            "grant_type": "client_credentials"
        }

        for attempt in range(MAX_RETRIES):
            try:
                response = self.session.post(url, data=data, timeout=10)
                if response.status_code == 200:
                    logger.info("🔑 认证成功, 获取 Token。")
                    return response.json()["access_token"]
                else:
                    logger.warning(f"⚠️ Token 获取失败 (尝试 {attempt+1}/{MAX_RETRIES}): {response.status_code}")
                    time.sleep(RETRY_DELAY)
            except Exception as e:
                logger.warning(f"⚠️ 网络异常 (尝试 {attempt+1}/{MAX_RETRIES}): {str(e)}")
                time.sleep(RETRY_DELAY)

        logger.critical("❌ 无法获取访问令牌。")
        return None

    def get_all_users(self) -> Optional[List[Dict]]:
        if not self.token: return None

```

```

        users = []
        url = "https://graph.microsoft.com/v1.0/users"
        headers = {"Authorization": f"Bearer {self.token}", "ConsistencyLevel":
"eventual"}
        params = {"$filter": "accountEnabled eq true", "$top": 999, "$select":
"id,userPrincipalName,displayName,usageLocation,assignedLicenses"}

        logger.info("🔄 正在获取所有启用的用户...")

        while url:
            try:
                response = self.session.get(url, headers=headers, params=params,
timeout=10)

                if response.status_code != 200:
                    logger.error(f"❌ 获取用户失败: {response.status_code}")
                    return None

                data = response.json()
                users.extend(data.get('value', []))
                url = data.get('@odata.nextLink')
                params = {}
            except Exception as e:
                logger.error(f"❌ 请求用户列表时发生异常: {str(e)}")
                return None

        logger.info(f"✅ 成功获取 {len(users)} 个启用的用户。")
        return users

    def get_group_id(self, group_name: str) -> Optional[str]:
        if not self.token: return None

        url = "https://graph.microsoft.com/v1.0/groups"
        headers = {"Authorization": f"Bearer {self.token}", "ConsistencyLevel":
"eventual"}
        params = {"$filter": f"displayName eq '{group_name}'", "$select": "id"}

        try:
            response = self.session.get(url, headers=headers, params=params,
timeout=10)

            if response.status_code == 200:
                groups = response.json().get('value', [])
                if groups: return groups[0]['id']
            else:
                logger.warning(f"⚠️ 组 '{group_name}' 未找到。")
                return None
        except Exception as e:
            logger.error(f"❌ 查询组 ID 失败: {response.status_code}")
            return None
        except Exception as e:
            logger.error(f"❌ 查询组 ID 时发生异常: {str(e)}")
            return None

    def get_group_members(self, group_id: str) -> List[str]:
        members = []
        if not self.token: return members

```

```

        url = f"https://graph.microsoft.com/v1.0/groups/{group_id}/members?
$select=id"
        headers = { "Authorization": f"Bearer {self.token}" }

        try:
            while url:
                response = self.session.get(url, headers=headers, timeout=10)
                if response.status_code == 200:
                    data = response.json()
                    members.extend([m['id'] for m in data['value']])
                    url = data.get('@odata.nextLink')
                else:
                    logger.error(f"✗ 获取组成员失败: {response.status_code}")
                    break
        except Exception as e:
            logger.error(f"✗ 获取组成员时发生异常: {str(e)}")

        return members

    def modify_membership(self, group_id: str, user_id: str, action: str = "add") ->
bool:
        if not self.token: return False

        if action == "add":
            url = f"https://graph.microsoft.com/v1.0/groups/{group_id}/members/$ref"
            method = self.session.post
            data = {"@odata.id": f"https://graph.microsoft.com/v1.0/directoryObjects/
{user_id}"}
        else:
            url = f"https://graph.microsoft.com/v1.0/groups/{group_id}/members/
{user_id}/$ref"
            method = self.session.delete
            data = None

        try:
            for attempt in range(MAX_RETRIES):
                response = method(url, headers={"Authorization": f"Bearer
{self.token}"}, json=data)

                if response.status_code in [200, 204]:
                    return True
                elif response.status_code == 429:
                    wait = int(response.headers.get('Retry-After', RETRY_DELAY)) + 1
                    logger.warning(f"⚠ 请求过多, 等待 {wait} 秒...")
                    time.sleep(wait)
                else:
                    logger.error(f"✗ {action.upper()} 失败: {response.status_code}")
                    time.sleep(RETRY_DELAY)
                    break
            return False
        except Exception as e:
            logger.error(f"✗ {action.upper()} 异常: {str(e)}")
            return False

    def run_sync(self):

```

```

start_time = datetime.now()
logger.info(f"🚀 同步任务开始 | 时间: {start_time}")

self.token = self.get_app_token()
if not self.token: sys.exit(1)

logger.info("🔍 解析目标组 ID...")
group_cache = {}
all_target_groups = list(COUNTRY_GROUP_MAP.values())
if DEFAULT_GROUP_NAME: all_target_groups.append(DEFAULT_GROUP_NAME)

for g_name in set(all_target_groups):
    g_id = self.get_group_id(g_name)
    if g_id:
        group_cache[g_name] = g_id
    else:
        logger.error(f"❌ 组 '{g_name}' 不存在, 跳过。")

if not group_cache:
    logger.critical("❌ 无有效目标组, 退出。")
    sys.exit(1)

all_users = self.get_all_users()
if not all_users: sys.exit(1)

licensed_users = [u for u in all_users if u.get("assignedLicenses")]
logger.info(f"📋 用户统计 | 总计: {len(all_users)} | 有License: {len(licensed_users)}")

target_memberships = {}
for user in licensed_users:
    u_id = user['id']
    u_loc = user.get('usageLocation')
    target_group_name = None

    if u_loc and u_loc in COUNTRY_GROUP_MAP:
        target_group_name = COUNTRY_GROUP_MAP[u_loc]
    elif DEFAULT_GROUP_NAME and (not u_loc or u_loc not in COUNTRY_GROUP_MAP):
        target_group_name = DEFAULT_GROUP_NAME

    if target_group_name and target_group_name in group_cache:
        if target_group_name not in target_memberships:
            target_memberships[target_group_name] = []
        target_memberships[target_group_name].append(u_id)

stats = {"added": 0, "removed": 0, "errors": 0}

for group_name, target_user_ids in target_memberships.items():
    group_id = group_cache[group_name]
    current_member_ids = self.get_group_members(group_id)

    to_add = set(target_user_ids) - set(current_member_ids)
    to_remove = set(current_member_ids) - set(target_user_ids)

    logger.info(f"🔄 组 '{group_name}' | 待添加: {len(to_add)} | 待移除: ")

```

```

{len(to_remove)}")

        for user_id in to_add:
            if self.modify_membership(group_id, user_id, "add"): stats['added']
+= 1
            else: stats['errors'] += 1

        for user_id in to_remove:
            if self.modify_membership(group_id, user_id, "remove"):
stats['removed'] += 1
            else: stats['errors'] += 1

    duration = (datetime.now() - start_time).total_seconds()
    summary = (
        f"\n{'='*50}\n"
        f"📊 任务完成总结\n"
        f"{'='*50}\n"
        f"🕒 运行时长: {duration:.2f} 秒\n"
        f"➕ 成功添加: {stats['added']}\n"
        f"➖ 成功移除: {stats['removed']}\n"
        f"❌ 操作错误: {stats['errors']}\n"
        f"✅ 任务状态: {'成功' if stats['errors'] == 0 else '部分失败'}"
    )

    if stats['errors'] == 0: logger.info(summary)
    else: logger.error(summary)

def main():
    syncer = AzureADSync()
    try:
        syncer.run_sync()
    except KeyboardInterrupt:
        logger.warning("🛑 任务被中断。")
        sys.exit(0)
    except Exception as e:
        logger.critical(f"💣 任务崩溃: {str(e)}", exc_info=True)
        sys.exit(1)

if __name__ == "__main__":
    main()

```